

Path Replanning

Path Replanning

1. Course Content
2. Run Path Replanning Case
 - 2.1 Start Sensors and Odometry
 - 2.3 Start Road Network Planning and Navigation Node
3. Query Obstacle Cost Threshold
 - 3.1 View Obstacle Cost Value
 - 3.2 Modify the Threshold for Triggering Replanning
4. Source Code Analysis
 - 4.1 Obstacle Detection and Replanning Strategy
 - 4.2 Method for Querying Map Cost Value

1. Course Content

- Basic

Run the sample program. Based on road network planning and navigation, if the driving path is detected to be blocked during travel, it will enter replanning to find a new path in the road network.

- Advanced

1. Understand the implementation method of replanning and how to change the replanning strategy (the strategy is not unique and can be implemented according to your own needs).
2. Debug and change the obstacle detection threshold based on the queried cost value.

This section requires rebuilding the map using a wider and more complex terrain to facilitate replanning.

2. Run Path Replanning Case

2.1 Start Sensors and Odometry

- Open a terminal on the host machine to start the radar fusion node and odometry fusion of the vehicle.

```
ros2 launch yahboomcar_nav laser_odom_bringup_launch.py
```

```

root@raspberrypi:~/yahboomcar_ros2_ws/yahboomcar_ws# ros2 launch yahboomcar_nav laser_odom_bringup.launch.py
[INFO] [launch]: All log files can be found below /root/.ros/log/2026-01-08-01-34-17-442171-raspberrypi-432
[INFO] [launch]: Default logging verbosity is set to INFO
----- rplidar_type = 4R0S -----

----- rplidar_type = 4R0S -----

[INFO] [ydlidar_ros2_driver_node-4]: process started with pid [449]
[INFO] [joint_state_publisher-1]: process started with pid [443]
[INFO] [robot_state_publisher-2]: process started with pid [445]
[INFO] [imu_driver-3]: process started with pid [447]
[INFO] [rf2o_laser_odometry_node-5]: process started with pid [451]
[ydlidar_ros2_driver_node-4] [YDLIDAR INFO] Current ROS Driver Version: 1.0.1
[ydlidar_ros2_driver_node-4]
[ydlidar_ros2_driver_node-4] YDLidar SDK initializing
[ydlidar_ros2_driver_node-4] YDLidar SDK has been initialized
[ydlidar_ros2_driver_node-4] [YDLIDAR]:SDK Version: 1.0.6
[rf2o_laser_odometry_node-5] Publisher already registered for provided node name. If this is due to multiple nodes with the same name then all logs for that l
ogger name will go out over the existing publisher. As soon as any node with that name is destructed it will unregister the publisher, preventing any further
logs for that name from being published on the rosout topic.
[robot_state_publisher-2] The root link base link has an inertia specified in the URDF, but KDL does not support a root link with an inertia. As a workaround
, you can add an extra dummy link to your URDF.
[robot_state_publisher-2] got segment base_footprint
[robot_state_publisher-2] got segment base_link
[robot_state_publisher-2] got segment camera_link
[robot_state_publisher-2] got segment imu_link
[robot_state_publisher-2] got segment laser
[robot_state_publisher-2] got segment yh1_Link
[robot_state_publisher-2] got segment yh2_Link
[robot_state_publisher-2] got segment yh3_Link
[robot_state_publisher-2] got segment yq1_Link
[robot_state_publisher-2] got segment yq2_Link
[robot_state_publisher-2] got segment yq3_Link
[robot_state_publisher-2] got segment yz1_Link
[robot_state_publisher-2] got segment yz2_Link
[robot_state_publisher-2] got segment yz3_Link
[robot_state_publisher-2] got segment zh1_Link
[robot_state_publisher-2] got segment zh2_Link
[robot_state_publisher-2] got segment zh3_Link
[robot_state_publisher-2] got segment zq1_Link
[robot_state_publisher-2] got segment zq2_Link
[robot_state_publisher-2] got segment zq3_Link
[robot_state_publisher-2] got segment zz1_Link
[robot_state_publisher-2] got segment zz2_Link
[robot_state_publisher-2] got segment zz3_Link
[rf2o_laser_odometry_node-5] Initializing RF20 node...
[rf2o_laser_odometry_node-5] Waiting for laser_scans....
[rf2o_laser_odometry_node-5] Waiting for laser_scans....
[rf2o_laser_odometry_node-5] Waiting for laser_scans....
[ydlidar_ros2_driver_node-4] LIDAR successfully connected
[rf2o_laser_odometry_node-5] Waiting for laser_scans....
[rf2o_laser_odometry_node-5] Waiting for laser_scans....
[rf2o_laser_odometry_node-5] Waiting for laser_scans....
[joint_state_publisher-1] Waiting for robot_description to be published on the robot_description topic...
[rf2o_laser_odometry_node-5] Waiting for laser_scans....

```

2.3 Start Road Network Planning and Navigation Node

- Start the Kilted container

```
~/kilted_docker_start.sh
```

- Start the road network planning and navigation node

```

ros2 launch road_net_route roadnet_nav2.launch.py
map:=/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/road_map.yaml
pose_map:=/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/road_map
roadnet_file:=/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/roa
d_map.geojson

```

[!TIP]

Optional startup parameter description

map The name of the grid map file to be loaded, the default value is `map.yaml`

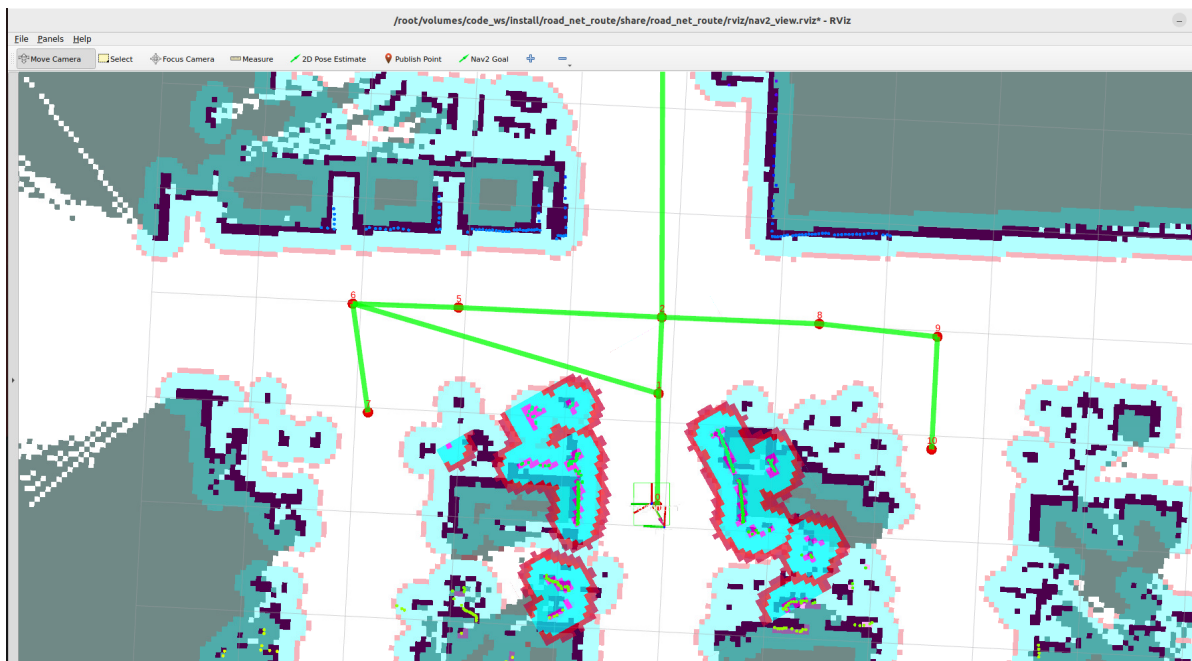
pose_map The name of the pose map file to be loaded. When using slam_toolbox pure localization for global navigation, a pose map must be provided.

params_file Navigation parameter file, the default value is

```
os.path.join(get_package_share_directory('road_net_route'), 'config',
'nav2_kilted.yaml')
```

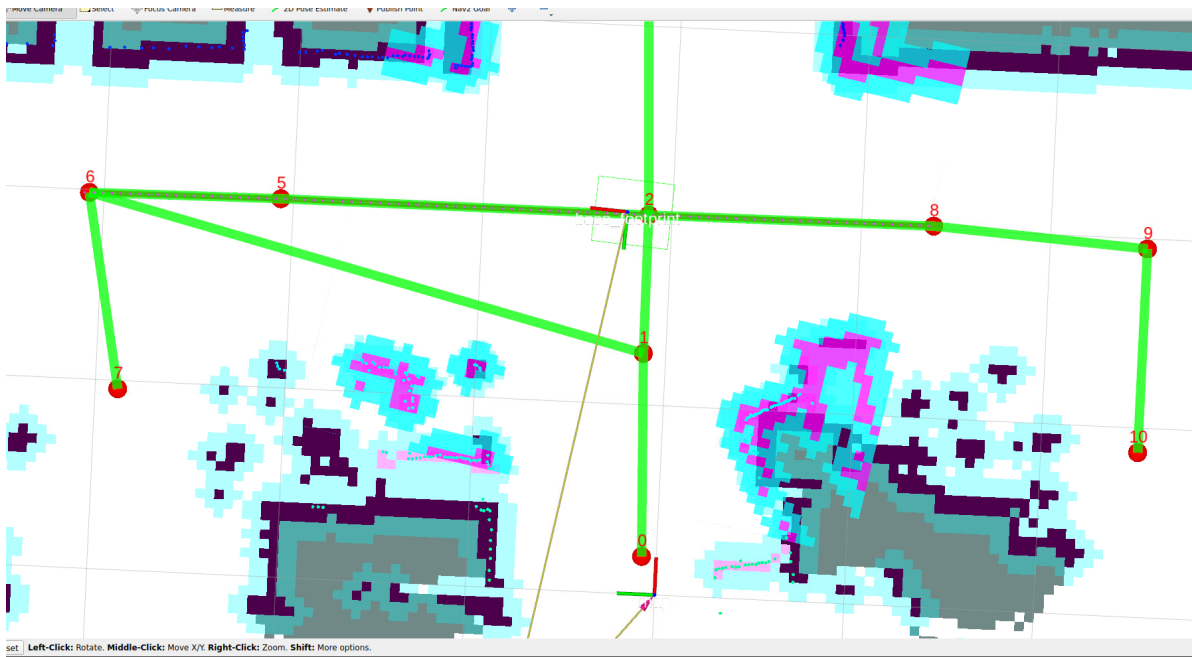
roadnet_file Road network description file, the default value is `/root/map/map.geojson`, a full file path must be provided here.

localization_mode Global localization method, the default is `slam_toolbox`



- The test scenario here is: travel from waypoint 8 to waypoint 6 along the road network, then place an obstacle on the planned path of the road network to trigger the replanning strategy.

```
ros2 topic pub -1 /road_net_nav_id std_msgs/msg/Int16MultiArray "{data: [8, 6]}"
```



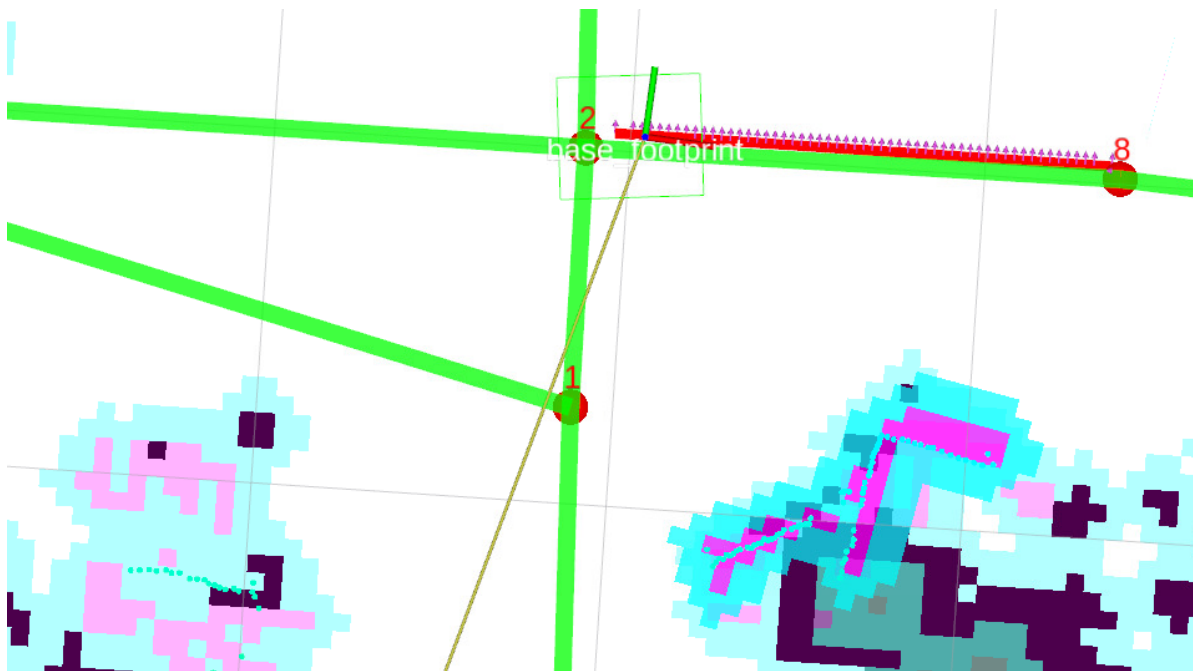
- Here you can see that the terminal will prompt that there is an obstacle on the path, and then trigger replanning.

```

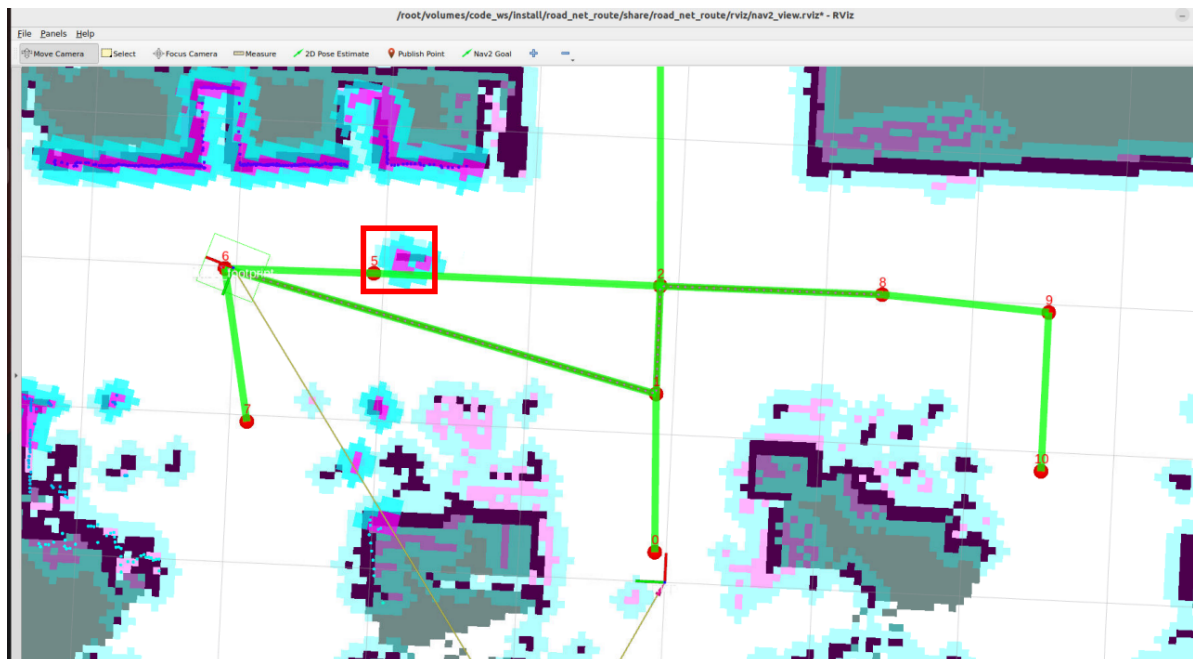
Rate of 20.0000 Hz. Current loop rate is 13.7802 Hz.
[route_bridge-4] [INFO] [1764248055.001985898] [route_bridge]: id:2 -----> id:5 edge:16
[component_container_isolated-1] [INFO] [1764248055.687587193] [global_costmap.global_costmap]: StaticLayer: Resizing costmap to 225 X 599 at 0.050000 m/pix
[route_bridge-4] [INFO] [1764248057.758462756] [route_bridge]: Path contains 7 potential obstacles.
[route_bridge-4] [INFO] [1764248058.249064867] [route_bridge]: Navigating to goal: 2.0027759075164795 0.1238360032 4392319...

```

- After detecting an obstacle on the path and the path is not passable, it returns to the last passed waypoint 8.



- Then, it tries to plan a drivable path again from waypoint 8 and then reaches the target waypoint 6.



3. Query Obstacle Cost Threshold

- The detection of obstacles on the path is determined by checking the cost value on the global path and the number of high-cost points in the global path. The threshold for triggering replanning can be set by yourself.

[!NOTE]

- The obstacle detection threshold is generally fine with the factory default!!! If you have special scene requirements, you can refer to this part of the tutorial to adjust the threshold for triggering replanning.
- The cost value is a value in the cost map used to describe the occupied range of an obstacle. The closer to the obstacle, the higher the cost value.

3.1 View Obstacle Cost Value

- Click the Publish Point tool, and then left-click on the map where you want to view the map cost value to see the cost value of the specified location.



- You can see that the cost value is 0 where there are no obstacles, and the cost value at the center of the obstacle is 99-100.

```
e at time 1764249617.787 for reason: discarding Message because the queue is full
[route_bridge-4] [INFO] [1764249619.908913466] [route_bridge]: clicked_point in global_costmap cost:0
[route_bridge-4] [INFO] [1764249642.080688451] [route_bridge]: clicked_point in global_costmap cost:100
[route_bridge-4] [INFO] [1764249653.790774087] [route_bridge]: clicked_point in global_costmap cost:99
```

3.2 Modify the Threshold for Triggering Replanning

- Configuration file path

```
~/code_ws/src/road_net_route/config/nav2_kilted.yaml
```

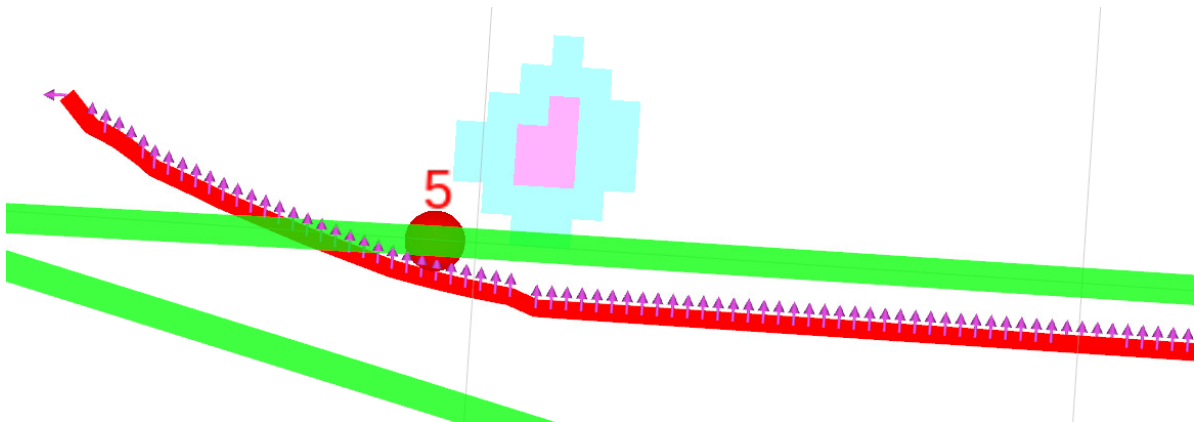
- Find the configuration parameters of the route_bridge node in the configuration file, where `COST_THRESHOLD` and `PATH_OBSTACLES_THRESHOLD` together determine the trigger threshold.

```
route_bridge:
  ros__parameters:
    roadnet_file: '/root/map/map.geojson'
    COST_THRESHOLD: 60
    replace2id: True
    PATH_OBSTACLES_THRESHOLD: 5
```

- The specific obstacle detection logic is as follows:
- The global path planned by the road network function is a `nav_msgs/msg/Path` message type in ROS, which is a path composed of multiple pose points. During road network navigation, it will detect whether all pose points on the global path have high cost values in the global cost map.
 - If the cost value of a certain pose point on the path exceeds the **COST_THRESHOLD** threshold, the point on the path is marked as a high-cost pose point.
 - Finally, it calculates how many pose points in the global path are high-cost pose points.
 - If the total number of high-cost pose points is greater than **PATH_OBSTACLES_THRESHOLD**, the path is considered impassable and needs to be replanned.

[!TIP]

If the obstacle detection is not sensitive, you can reduce **COST_THRESHOLD** and **PATH_OBSTACLES_THRESHOLD**. If it is too sensitive, you can increase these two values or adjust the inflation radius of the obstacle (to modify the obstacle inflation radius, refer to 4.1 Navigation Controller Debugging in 04-Inter-Road-Network-Planning-and-Navigation).



4. Source Code Analysis

- Implementation source code path:

```
~/code_ws/src/road_net_route/road_net_route/route_bridge.py
```

4.1 Obstacle Detection and Replanning Strategy

- Traverse each pose point of the global path, check for obstacles exceeding the threshold on the path through the global costmap, and return a boolean value indicating whether the path is blocked by obstacles.
- **World to Map Coordinates:** Call the costmap's `worldToMapValidated` method to convert world coordinates `(wx, wy)` to the costmap's **grid coordinates** `(mx, my)` (the costmap is a 2D grid map, where each grid corresponds to a cost value).
- **Coordinate Validity Check:** If the converted grid coordinates are `None`, it means the path point is outside the global costmap range. A warning log is printed, and the point is skipped.
- **Parameter:** `path:Path` → ROS2's `nav_msgs/msg/Path` message type, which stores the robot's global planned path, containing a series of poses (`poses`).
- **Return Value:** `bool` → `True` indicates the number of obstacles on the path exceeds the threshold; `False` indicates the path is feasible.

```
def __check_path_obstacles(self, path:Path)->bool:
    """Check for obstacles on the global path."""
    pose_array=[]
    for pose in path.poses:
        wx=pose.pose.position.x
        wy=pose.pose.position.y

        mx, my = self.global_costmap.worldToMapValidated(wx,wy )
        if mx is None or my is None:
            self.get_logger().warn(f"Path point ({wx}, {wy}) is outside global
costmap")
            continue

        cost = self.global_costmap.getCostXY(int(mx),int(my))

        if cost > self.COST_THRESHOLD:
            pose_array.append(pose)
    if len(pose_array) >self.PATH_OBSTACLES_THRESHOLD:
        self.get_logger().info(f"Path contains {len(pose_array)} potential
obstacles.")
        return True
    return False
```

- When an obstacle is detected on the global path, make the robot **retreat to the previous waypoint** and replan the path.

```
def __reroute_strategy_1(self,last_id):
    '''Replanning Strategy 1: Go back to the previous waypoint and replan the
drivable path.'''
    navigator= BasicNavigator()
    goal_x,goal_y=self.__get_node_coordinates(last_id)
    if goal_x is not None and goal_y is not None:
        goal_pose = PoseStamped()
        goal_pose.header.frame_id = "map"
        goal_pose.pose.position.x = goal_x
        goal_pose.pose.position.y = goal_y
        back_lastid = navigator.goToPose(goal_pose)
        while not navigator.isTaskComplete(task=back_lastid):
```



```

        time.sleep(0.1)
    if navigator.getResult() == TaskResult.SUCCEEDED:
        self.get_logger().info("back to last id success.")
    else:
        self.get_logger().info("back to last id failed.")

```

- `handle_id_navigation` and `handle_pose_navigation` are the core code responsible for road network navigation. The following part in both functions is the logic for detecting the path and triggering replanning.

When an impassable path is detected, **Replanning Strategy 1** is called: make the robot **retreat to the previous waypoint** (coordinates corresponding to `last_node_id`). The core logic of this method has been explained before: initialize the navigator → get the coordinates of `last_node_id` → send a navigation task to "retreat to this waypoint" → wait for the task to complete and print the result. This moves the robot from the current obstacle area back to a **safe predefined waypoint**, providing a stable starting point for subsequent path replanning.

The complete process is as follows:

1. The robot normally executes path tracking and gets real-time navigation feedback `feedback`.
2. Check if there are obstacles in the path from `feedback`:
 - If no obstacles: continue on the original path, no action needed.
 - If there are obstacles: trigger the replanning logic.
3. The robot retreats to the last safe waypoint (`last_node_id`).
4. Starting from this waypoint, replan the path to the target waypoint (`goal_id`).
5. Get feedback for the new path and have the robot follow the new path to continue moving.

```

if feedback is not None and self.__check_path_obstacles(feedback.path):
    self.__reroute_strategy_1(last_node_id)
    route_task = self.navigator.getAndTrackRoute(last_node_id, goal_id)
    feedback = self.navigator.getFeedback(task=route_task)
    follow_path_task = self.navigator.followPath(feedback.path)

```

```

def handle_id_navigation(self, start_id:int, goal_id:int):
    '''路点间导航road network ID navigation'''

    route_task = self.navigator.getAndTrackRoute(start_id, goal_id)
    feedback=None
    follow_path_task=RunningTask.NONE
    last_feedback = None
    last_node_id =999
    while not self.navigator.isTaskComplete(task=route_task):
        feedback = self.navigator.getFeedback(task=route_task)
        if feedback is not None and
self.__check_path_obstacles(feedback.path):
            self.__reroute_strategy_1(last_node_id)

```



```

        route_task = self.navigators.getAndTrackRoute(last_node_id,
goal_id)

        feedback = self.navigators.getFeedback(task=route_task)
        follow_path_task = self.navigators.followPath(feedback.path)
        while feedback is not None:
            if last_feedback is not None and (feedback.last_node_id !=
last_feedback.last_node_id or feedback.next_node_id !=
last_feedback.next_node_id):
                self.get_logger().info(f'id:{feedback.last_node_id} ----->
id:{feedback.next_node_id} edge:{feedback.current_edge_id}')
                last_feedback = feedback
                last_node_id=feedback.last_node_id
            if feedback.rerouted :
                self.get_logger().info('reroute a new path to follow!')
                follow_path_task = self.navigators.followPath(feedback.path)
                feedback = self.navigators.getFeedback(task=route_task)

        if self.navigators.isTaskComplete(task=follow_path_task):
            print(follow_path_task)
            self.get_logger().info('Controller completed its task!')
            if follow_path_task != RunningTask.NONE :
                self.navigators.cancelTask()

        result = self.navigators.getResult()
        if result == TaskResult.SUCCEEDED:
            self.get_logger().info(Fore.GREEN+"navigation task
completed."+Fore.RESET)

        self.action_feedback_pub.publish(String(data="road_net_nav_succeeded"))
        elif result == TaskResult.CANCELED:
            self.get_logger().warn("Navigation task canceled.")
        elif result == TaskResult.FAILED:
            self.get_logger().error("Navigation task failed.")
            self.action_feedback_pub.publish(String(data="road_net_nav_failed"))

    def handle_pose_navigation(self, goal_pose:PoseStamped):
        '''处理任意位姿导航 arbitrary pose navigation'''
        transform = self.tf_buffer.lookup_transform("map", "base_footprint",
rclpy.time.Time(),rclpy.duration.Duration(seconds=5.0))
        if transform is None:
            self.get_logger().info(Fore.RED+"Failed to get TF transform
"+Fore.RESET)
            return

        else:
            start_pose = PoseStamped()
            start_pose.header = transform.header
            start_pose.pose.position.x = transform.transform.translation.x
            start_pose.pose.position.y = transform.transform.translation.y
            start_pose.pose.orientation = transform.transform.rotation
            self.get_logger().info(Fore.CYAN + f"current:
{transform.transform.translation}"+Fore.RESET)
            nearest_start_node_id, nearest_start_distance =
self.__find_nearest_node(start_pose.pose.position.x, start_pose.pose.position.y)
            nearest_goal_node_id, nearest_goal_distance =
self.__find_nearest_node(goal_pose.pose.position.x, goal_pose.pose.position.y)
            if self.replace2id:

```

```

        self.get_logger().info(Fore.CYAN + f"nearest_start_node_id:
{nearest_start_node_id}, distance:{nearest_start_distance}"
                                f"\nearest_goal_node_id:
{nearest_goal_node_id}, distance:{nearest_goal_distance}" + Fore.RESET)
        route_task =
self.navigator.getAndTrackRoute(start=nearest_start_node_id,goal=nearest_goal_no
de_id)

    else:
        route_task =
self.navigator.getAndTrackRoute(start=start_pose,goal=goal_pose,use_start=True)

        feedback=None
        follow_path_task=RunningTask.NONE
        last_feedback = None
        last_node_id =999
        while not self.navigator.isTaskComplete(task=route_task):
            feedback = self.navigator.getFeedback(task=route_task)
            if feedback is not None and
self.__check_path_obstacles(feedback.path):
                self.__reroute_strategy_1(last_node_id)
                route_task = self.navigator.getAndTrackRoute(last_node_id,
nearest_goal_node_id)
                feedback = self.navigator.getFeedback(task=route_task)
                follow_path_task = self.navigator.followPath(feedback.path)
                while feedback is not None:
                    if last_feedback is not None and (feedback.last_node_id !=
last_feedback.last_node_id or feedback.next_node_id !=
last_feedback.next_node_id):
                        self.get_logger().info(f'id:{feedback.last_node_id} ----
--> id:{feedback.next_node_id} edge:{feedback.current_edge_id}')
                        last_feedback = feedback
                        last_node_id=feedback.last_node_id
                        if feedback.rerouted :
                            self.get_logger().info('reroute a new path to follow!')
                            follow_path_task =
self.navigator.followPath(feedback.path)
                            feedback = self.navigator.getFeedback(task=route_task)

                    if self.navigator.isTaskComplete(task=follow_path_task):
                        self.get_logger().info('Controller completed its task!')
                        if follow_path_task != RunningTask.NONE :
                            self.navigator.cancelTask()

                result = self.navigator.getResult()
                if result == TaskResult.SUCCEEDED:
                    if self.__adjust_pose(goal_pose):
                        self.get_logger().info(Fore.GREEN+"navigation task
completed." +Fore.RESET)

                        self.action_feedback_pub.publish(String(data="road_net_nav_succeeded"))

                    elif result == TaskResult.CANCELED:
                        self.get_logger().info(Fore.YELLOW+"navigation task
canceled." +Fore.RESET)

                    elif result == TaskResult.FAILED:
                        self.get_logger().info(Fore.RED+"navigation task
failed." +Fore.RESET)

```

```
self.action_feedback_pub.publish(String(data="road_net_nav_failed"))
```

4.2 Method for Querying Map Cost Value

- Call the global costmap's `worldToMapValidated` method to convert **world coordinates** `(x, y)` to the **costmap's grid coordinates** `(mx, my)`.
- The costmap is a **2D grid map** (each grid corresponds to a small area in the real world, 0.05m×0.05m). `worldToMapValidated` is a safe conversion method: if the coordinates are outside the costmap range, it returns `None`, avoiding out-of-bounds errors from direct conversion.
- **Get Cost Value:** Call the costmap's `getCostXY` method, passing the integer form of the grid coordinates (grid coordinates are discrete integers, avoiding errors from floating-point input), to get the **cost value** `cost` of that grid.

```
def get_costmap_value(self, msg: PointStamped):  
    '''In RViz, you can click to view the cost of a point on the global cost  
map.'''  
    x=msg.point.x  
    y=msg.point.y  
    mx, my = self.global_costmap.worldToMapValidated(x, y)  
    if mx is None or my is None:  
        self.get_logger().warn(f"Path point ({x}, {y}) is outside local  
costmap")  
        return  
    cost = self.global_costmap.getCostXY(int(mx),int(my))  
    self.get_logger().info(f"clicked_point in global_costmap cost:{cost}")
```